

Terraform Blocks - Class 22 Notes

Introduction

Terraform is an Infrastructure as Code (IaC) tool used to automate the provisioning and management of cloud resources across platforms such as Azure, AWS, GCP, and Oracle Cloud.

Terraform uses **Providers** to communicate with cloud platforms and **Blocks** to define infrastructure configurations.

Terraform Architecture

```
graph TD;
    Provider --> Terraform_exe[Terraform.exe];
    Terraform_exe --> Plugin;
    Plugin --> Cloud_Platform[Cloud Platform];
```

Example

```
graph TD;
    AzureRM_Provider[AzureRM Provider] --> Terraform;
    Terraform --> Azure_Plugin[Azure Plugin];
    Azure_Plugin --> Azure_Cloud[Azure Cloud];
```

```
graph TD;
    AWS_Provider[AWS Provider] --> Terraform;
    Terraform --> AWS_Plugin[AWS Plugin];
    AWS_Plugin --> AWS_Cloud[AWS Cloud];
```

Terraform Configuration File

Terraform code is stored in `.tf` files.

Examples:

```
main.tf
provider.tf
network.tf
vm.tf
```

Terraform automatically reads all `.tf` files present in the working directory.

Terraform Blocks

Terraform configuration is written using blocks.

Generic Syntax

```
Block_Type {
    arguments = values
}
```

Example:

```
terraform {
}
```

Arguments and Attributes

Arguments are configurations defined inside a block.

Required Arguments

Mandatory values required by Terraform.

Example:

```
name
location
```

Optional Arguments

Additional settings that can be omitted.

Example:

```
tags
timeouts
description
```

Terraform Data Types

String

```
name = "CloudTechHacks"
```

Number

```
vm_count = 2
```

Boolean

```
enable_backup = true
```

List (Array)

```
colors = ["red","green","blue"]
```

Map

```
tags = {
  Environment = "Production"
  Owner       = "Admin"
}
```

JSON vs Terraform Map

JSON

```
{
  "Name": "Server1",
  "Location": "India"
}
```

Terraform Map

```
{
  Name      = "Server1"
  Location  = "India"
}
```

Difference

JSON	Terraform
Uses :	Uses =
Key in quotes	Quotes optional
Data format	Configuration format

Terraform Block Types

1. Terraform Block

Used to configure Terraform settings.

```
terraform {
  required_version = ">=1.5.0"
}
```

Purpose

- Terraform Version Control
- Backend Configuration
- Required Providers

2. Provider Block

Used to connect Terraform with a cloud provider.

Syntax

```
provider "azurerm" {  
  features {}  
}
```

Explanation

Component	Description
provider	Block Type
azurerm	Provider Name
features {}	Mandatory AzureRM configuration

3. Resource Block

Used to create infrastructure resources.

Syntax

```
resource "RESOURCE_TYPE" "LOCAL_NAME" {  
  
}
```

Example:

```
resource "azurerm_resource_group" "rg" {  
  name      = "prod-rg"  
  location = "Central India"  
}
```

Explanation

Component	Description
resource	Block Type
azurerm_resource_group	Resource Type
rg	Local Name

Component	Description
name	Resource Name
location	Azure Region

Azure Resource Blocks

Resource Group

```
resource "azurerm_resource_group" "rg" {
  name      = "prod-rg"
  location  = "Central India"
}
```

Virtual Network

```
resource "azurerm_virtual_network" "vnet" {
  name            = "prod-vnet"
  address_space   = ["10.0.0.0/16"]
  location        = "Central India"
  resource_group_name = azurerm_resource_group.rg.name
}
```

Virtual Machine

```
resource "azurerm_linux_virtual_machine" "vm" {
}
```

Common Azure Resource Types

Terraform Resource	Azure Resource
azurerm_resource_group	Resource Group
azurerm_virtual_network	Virtual Network
azurerm_subnet	Subnet

Terraform Resource	Azure Resource
azurerm_network_interface	NIC
azurerm_public_ip	Public IP
azurerm_linux_virtual_machine	Linux VM
azurerm_windows_virtual_machine	Windows VM
azurerm_storage_account	Storage Account

Terraform Workflow

Write Terraform Code

↓

`terraform init`

↓

Download Providers

↓

`terraform validate`

↓

Validate Syntax

↓

`terraform plan`

↓

Preview Changes

↓

`terraform apply`

↓

Create Infrastructure

Important Terraform Commands

Initialize Project

```
terraform init
```

Downloads provider plugins.

Validate Configuration

```
terraform validate
```

Checks syntax and configuration errors.

Generate Execution Plan

```
terraform plan
```

Shows what Terraform will create, update, or destroy.

Apply Configuration

```
terraform apply
```

Creates infrastructure resources.

Destroy Infrastructure

```
terraform destroy
```

Deletes all managed resources.

Interview Questions

What is a Terraform Block?

A Terraform Block is the basic building unit of Terraform configuration used to define providers, resources, variables, outputs, and settings.

What is a Provider?

A Provider is a plugin that enables Terraform to interact with cloud platforms such as Azure, AWS, and GCP.

What is a Resource Block?

A Resource Block is used to create and manage cloud resources.

Why is `features {}` required in AzureRM Provider?

The AzureRM provider requires the `features {}` block for initialization and configuration.

What is the Local Name in a Resource Block?

Example:

```
resource "azurerm_resource_group" "rg" {  
}
```

`rg` is the local name used to reference the resource within Terraform code.

Quick Revision

Terraform Configuration = .tf File

Terraform Block = Configure Terraform

Provider Block = Connect to Cloud

Resource Block = Create Infrastructure

String = "Text"

List = ["red","green","blue"]

```
Map = {  
  key = value  
}
```

terraform init = Download Providers

terraform validate = Validate Code

terraform plan = Preview Changes

terraform apply = Create Resources

terraform destroy = Delete Resources